PROVISIONAL PATENT APPLICATION

System and Method for Private Decentralized AI Inference on Adversarial Consumer Hardware via Software Access Path Elimination and Multi-Layer Hardware Attestation

Inventor: Gajesh Naik

Eigen Labs

gajesh@eigenlabs.org

March 2026

INVENTOR(S)

Gajesh Naik, Eigen Labs

1 FIELD OF THE INVENTION

The disclosed embodiments relate generally to distributed computing systems for artificial intelligence inference, and more particularly to systems and methods for enabling private, confidential AI computation on decentralized consumer hardware where the hardware owner is assumed to be adversarial, using software access path elimination techniques and multi-layer hardware attestation without requiring hardware-based Trusted Execution Environments (TEEs) for memory encryption.

2 CROSS-REFERENCE TO RELATED APPLICATIONS

Not applicable. This is an original provisional patent application and does not claim the benefit of any prior-filed application.

3 BACKGROUND OF THE INVENTION

3.1 Technical Problem

AI inference, particularly large language model (LLM) inference, requires substantial computational resources. Current approaches concentrate inference workloads on centralized cloud infrastructure using enterprise GPU hardware (e.g., NVIDIA H100 clusters), creating bottlenecks in cost, capacity, and availability.

A substantial quantity of underutilized compute exists in consumer hardware, particularly Apple Silicon Macs equipped with unified memory architectures and high-bandwidth GPU capabilities. Apple's M-series processors provide memory bandwidths ranging from 68 GB/s (M1) to 819 GB/s (M4 Ultra), with unified memory capacities up to 192 GB, sufficient for inference on models with billions of parameters.

However, utilizing third-party consumer hardware for inference presents a privacy challenge: the hardware owner has root access and physical custody of the machine. In conventional computing, the machine owner can inspect any process's memory, attach debuggers, intercept network traffic, and modify running software. Running confidential inference—where a user's prompts and the model's responses must remain private—on a machine controlled by an adversary therefore presents a substantial technical challenge.

3.2 Limitations of Existing Approaches

Hardware Trusted Execution Environments (TEEs). Intel TDX, AMD SEV-SNP, and NVIDIA Confidential Computing encrypt virtual machine memory and provide remote attestation. However, these solutions require enterprise server hardware costing $14,000 or more per unit and are unavailable on consumer hardware.

Apple Silicon Secure Enclave. While Apple Silicon includes a Secure Enclave co-processor, it provides only key generation and cryptographic signing services. It does not offer memory encryption or isolated execution environments for arbitrary third-party code. Model weights (up to 76 GB), tokenizer state, and intermediate activations cannot be placed within the Secure Enclave.

Cryptographic Approaches. Fully Homomorphic Encryption (FHE) introduces computational overhead of 10^4 to 10^6 times, making interactive LLM inference impractical. Secure Multi-Party Computation (MPC) requires multiple non-colluding servers. Zero-knowledge proof systems (e.g., zkLLM, DeepProve-1) verify inference integrity but do not protect input privacy during computation.

Existing Decentralized Compute Networks. Current decentralized compute networks (e.g., Akash, io.net, Ritual, Bittensor) either lack inference privacy guarantees entirely, rely on hardware TEEs unavailable on consumer devices, or use consensus mechanisms that verify results rather than protect inputs.

Apple Private Cloud Compute (PCC). Apple achieves inference privacy on Apple Silicon servers (M2 Ultra) in Apple-controlled data centers with physical security measures (access controls, tamper detection). However, PCC operates under a fundamentally different threat model where Apple owns and physically secures the hardware. PCC's approach cannot be directly applied to a decentralized network where individual hardware owners are assumed adversarial.

3.3 Need in the Art

There exists a need for a system and method that enables private AI inference on decentralized consumer hardware, where the hardware owner is adversarial, without requiring hardware memory encryption or trusted execution environments, while achieving practical interactive inference speeds.

4 SUMMARY OF THE INVENTION

Disclosed herein are systems and methods for private decentralized AI inference on consumer hardware controlled by adversarial third parties. Rather than encrypting memory (which consumer hardware does not support for arbitrary code), the disclosed systems and methods systematically identify and eliminate every software mechanism through which inference data—including user prompts and model outputs—could be observed by the hardware owner.

The disclosed systems and methods combine three principal aspects:

First, a software access path elimination architecture that leverages the operating system kernel's security mechanisms to create a hardened process in which AI inference executes. The hardened process is protected by: (a) anti-debugger attachment at the kernel level via the ptrace deny-attach mechanism, (b) hardened runtime enforcement that blocks external memory reading APIs, and (c) System Integrity

Protection (SIP) that prevents the hardware owner from disabling the foregoing protections without rebooting the machine—which terminates the inference process. A formal proof is provided that SIP provides runtime immutability: if SIP is verified as enabled at process startup, it remains enabled for the entire process lifetime, because the only mechanism to disable SIP requires a hardware reboot that terminates all running processes.

Second, a multi-layer attestation architecture comprising four independent verification layers that each verify different aspects of the provider's security posture: (1) Secure Enclave P-256 ECDSA signatures proving hardware-bound cryptographic identity; (2) Mobile Device Management (MDM) SecurityInfo queries that independently confirm security configuration through the operating system's management subsystem rather than through the provider's own software; (3) Apple Managed Device Attestation (MDA) producing Apple-signed X.509 certificate chains that prove genuine hardware identity and security state; and (4) continuous challenge-response verification at periodic intervals confirming that security posture has not degraded since enrollment. The disclosed embodiments further include a protocol for cryptographically binding a provider's Secure Enclave signing key to Apple-verified genuine hardware via an MDA nonce mechanism, circumventing platform keychain restrictions that otherwise prevent third-party applications from accessing hardware-attested keys.

Third, an in-process inference architecture that eliminates inter-process communication attack surfaces by embedding the AI inference engine directly within the hardened provider process via foreign function interface (FFI) bindings, rather than running it as a separate subprocess or local HTTP server. This eliminates attack vectors including localhost network traffic interception and subprocess binary replacement.

The system achieves inference privacy with approximately 12 milliseconds of performance overhead per request for security verification, while maintaining interactive speeds of 22 to 92 tokens per second depending on hardware and model size. The residual attack surface reduces to physical memory probing of LPDDR5x memory soldered directly into the Apple System-on-Chip package—a threat model comparable to that of Apple's Private Cloud Compute.

5 BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 is a block diagram illustrating the overall system architecture of the decentralized private inference system, showing the consumer, coordinator, provider, and attestation infrastructure components and their interconnections.

FIG. 2 is a flowchart illustrating the software access path elimination process executed at provider startup and per-request verification, showing the sequential security checks and their kernel-level enforcement mechanisms.

FIG. 3 is a block diagram illustrating the four-layer attestation architecture, showing the independent verification paths for Secure Enclave signatures, MDM SecurityInfo, Apple MDA certificate chains, and continuous challenge-response.

FIG. 4 is a sequence diagram illustrating the Secure Enclave key binding protocol via MDA nonce, showing the cryptographic binding of a provider's signing key to Apple-verified genuine hardware.

FIG. 5 is a block diagram illustrating the combined enrollment protocol using a single configuration profile containing SCEP, MDM, and ACME payloads.

FIG. 6 is a block diagram illustrating the in-process inference architecture, showing the FFI bridge between the Rust security layer and the Python inference engine within a single protected process.

FIG. 7 is a flowchart illustrating the provider scoring and selection algorithm, showing the six-factor composite scoring function with real-time hardware telemetry inputs.

FIG. 8 is a sequence diagram illustrating the complete request lifecycle, including end-to-end encryption, provider selection, decryption, inference execution, response transmission, and cancellation propagation.

FIG. 9 is a sequence diagram illustrating the periodic challenge-response protocol between the coordinator and provider.

FIG. 10 is a state diagram illustrating the idle GPU management lifecycle, including on-demand backend spawning, health monitoring, idle timeout, and lazy reload with exponential backoff.

6 DETAILED DESCRIPTION OF THE INVENTION

The following description sets forth numerous specific details to provide a thorough understanding of the disclosed embodiments. However, one skilled in the art will recognize that the disclosed embodiments may be practiced without some of these specific details. In some instances, well-known methods, procedures, components, and circuits have not been described in detail so as not to obscure the disclosed subject matter.

6.1 System Architecture (FIG. 1)

Referring now to FIG. 1, the decentralized private inference system **100** comprises four principal components: a consumer device **102**, a coordinator server **104**, one or more provider devices **106**, and attestation infrastructure **116**, **118**, **120**.

The consumer device **102** is any computing device that submits AI inference requests via a standard API interface. In one embodiment, the consumer device **102** communicates with the coordinator server **104** over HTTPS with TLS 1.3 encryption using an OpenAI-compatible API format.

The coordinator server **104** is a central matchmaking and verification server that routes inference requests from consumers **102** to providers **106**. In one embodiment, the coordinator server **104** runs within a hardware Trusted Execution Environment (e.g., Intel TDX) with cryptographic container image attestation, such that the coordinator's memory is hardware-encrypted. The coordinator server **104** comprises: a request router **128** that selects a provider for each inference request based on a multi-factor scoring algorithm; an attestation verifier **126** that verifies provider security posture through multiple independent channels; and a provider registry **130** that maintains the state, capabilities, and trust status of all connected providers.

Each provider device **106** is an Apple Silicon Mac with unified memory architecture and a Secure Enclave co-processor **108**. The provider device **106** runs a hardened inference agent process comprising: a security module **112** that enforces access path elimination; an inference engine **110** embedded directly within the process; and a cryptographic module that manages key generation and message decryption. The provider device **106** connects to the coordinator server **104** via a WebSocket connection **114** initiated by the provider (outbound connection), enabling operation behind Network Address Translation (NAT) and firewalls without requiring port forwarding.

The attestation infrastructure comprises: an MDM server **116** (e.g., MicroMDM with SCEP certificate issuance) for device enrollment and security information queries; an ACME Certificate Authority **118** (e.g., step-ca) configured for the `device-attest-01` challenge type; and Apple attestation servers **120** that participate in Managed Device Attestation by verifying device identity and issuing signed certificate chains.

6.2 Software Access Path Elimination (FIG. 2)

Referring now to FIG. 2, the software access path elimination method **200** operates to systematically block every software mechanism through which the hardware owner could observe inference data within the provider process. The method leverages three kernel-level protection mechanisms enforced by the operating system, plus architectural elimination of inter-process communication.

6.2.1 Anti-Debugger Attachment (Step 202)

At process startup, before any sensitive data is loaded, the provider process invokes the `ptrace` system call with the `PT_DENY_ATTACH` flag (system call number 31 on macOS). This kernel-level mechanism permanently denies all ptrace requests against the process for its entire lifetime, including requests from processes running as root. Specifically, this blocks: debugger attachment (e.g., lldb, dtrace), Instruments profiling, and any process tracing mechanism that relies on ptrace.

In one embodiment, the invocation is:

```
ptrace(PT_DENY_ATTACH, 0, NULL, 0)
```

This call is made before loading model weights or processing any inference requests, ensuring the process is hardened before it contains sensitive data.

6.2.2 Hardened Runtime Enforcement (Step 204)

The provider binary is code-signed with Hardened Runtime enabled and explicitly WITHOUT the `com.apple.securi` entitlement. The Hardened Runtime is enforced by the operating system kernel and provides the following protections: the kernel denies `task_for_pid()` calls targeting the provider process, which blocks Mach-level process inspection; the kernel denies `mach_vm_read()` and related virtual memory reading APIs from external processes; and code injection via `dlopen()` of unsigned libraries is blocked.

The absence of the `get-task-allow` entitlement is significant to the security model because this entitlement is the mechanism by which development tools (Xcode, lldb) are permitted to inspect processes. By omitting it from the production binary's entitlements, the kernel treats the process as a hardened target that cannot be inspected even by processes with elevated privileges.

6.2.3 System Integrity Protection Verification (Step 206)

System Integrity Protection (SIP) is an operating system security feature that prevents modification of protected system files and enforces the Hardened Runtime protections described above. When SIP is enabled, even the root user cannot: disable Hardened Runtime protections on signed binaries; load unsigned kernel extensions; modify protected system directories; or bypass the `PT_DENY_ATTACH` mechanism.

The provider process verifies that SIP is enabled at startup by executing `/usr/bin/csrutil status` and parsing the output for the string "enabled." If SIP is not enabled, the provider refuses to process inference requests.

Theorem 1 (SIP Runtime Immutability)**.** *Under the assumption that no unpatched kernel vulnerabilities exist, if SIP is verified as enabled at process startup time t_0, then SIP remains enabled at all times $t \in [t_0, t_{\text{end}}]$ during that process's lifetime.*

Proof. SIP state is stored in machine NVRAM and is immutable to userspace and kernel code in the normal macOS boot environment. The only mechanism to change SIP state requires: (1) rebooting into Recovery Mode; (2) executing the `csrutil disable` command; and (3) rebooting back to the normal environment. Step (1) requires a hardware reboot, which terminates every running process. Therefore, if the

provider process is alive and observed SIP as enabled at startup, SIP must still be enabled, because any attempt to disable it would have terminated the process via reboot. □

Corollary 1 (Single Verification Sufficiency)**.** *A single SIP verification at process startup provides a security guarantee for the entire process lifetime. Per-request SIP checks serve as defense-in-depth but are not strictly necessary under the conditions of the theorem.*

In one embodiment, the provider nonetheless performs per-request SIP verification as an additional defense-in-depth measure, adding approximately 12 milliseconds of overhead per request.

6.2.4 In-Process Inference Architecture (Step 208, FIG. 6)

Referring now to FIG. 6, the in-process inference architecture **600** eliminates inter-process communication attack surfaces by embedding the AI inference engine directly within the hardened provider process.

In conventional inference server architectures, the inference engine runs as a separate process (e.g., an HTTP server on localhost). This creates multiple attack vectors: localhost TCP traffic can be captured via network monitoring tools (e.g., `tcpdump`), which operates at the network layer and is not blocked by process-level protections even when SIP is enabled; the subprocess binary can be replaced with a malicious version that logs prompts and responses; inter-process shared memory regions can be read by other processes; and Unix domain sockets or named pipes can be intercepted.

In accordance with the disclosed embodiments, these attack vectors are eliminated by loading the inference engine directly into the provider process's address space via Foreign Function Interface (FFI) bindings. In one embodiment, a Rust-based security and networking layer hosts a Python interpreter via PyO3 FFI bindings, and the MLX framework is loaded within this embedded interpreter. Model weights (up to 76 GB), tokenizer state, and all intermediate activations reside within the single process's address space, protected by the kernel-level mechanisms described above.

6.2.5 Python Import Path Restriction (Step 608)

To prevent code injection via malicious Python packages, the system restricts the Python interpreter's import path at initialization. The provider executable locates its application bundle's `Frameworks/python` directory and sets `sys.path` to include only: (a) the bundled package directory within the code-signed application bundle; and (b) the Python standard library (excluding `site-packages`). The system `site-packages` directory is explicitly excluded.

This restriction is enforced by the following chain: the application bundle is code-signed; any modification to bundle contents invalidates the code signature; SIP prevents macOS from executing binaries with invalid code signatures. Therefore, the provider cannot inject malicious Python packages into the inference engine's import path without breaking the code signature, which prevents execution.

6.2.6 Memory Sanitization

After each inference request completes, all buffers containing prompts and model outputs are zeroed using volatile write operations that prevent compiler dead-store elimination:

```
for each byte in buffer:
    write_volatile(byte, 0)
sequential_consistency_memory_fence()
```

The volatile write ensures the compiler does not optimize away the zeroing operation, and the memory fence ensures the writes are committed to memory before the function returns.

6.2.7 Complete Attack Surface Analysis

The access path elimination method **200** blocks the following attack vectors:

Attack Vector	Defense Mechanism	Enforced By
Debugger attachment (lldb, dtrace)	`PT_DENY_ATTACH` at startup	Kernel
Memory reading via Mach APIs	Hardened Runtime w/o `get-task-allow`	Kernel
IPC interception	No IPC exists; inference in-process	Architecture
Provider binary modification	Code signing + SIP	Kernel + SIP
Binary replacement	Binary SHA-256 in SE-signed attestation	Coordinator
Malicious Python packages	`sys.path` locked to signed bundle	Process + SIP
Unsigned kernel extensions	SIP blocks all unsigned kexts	SIP
Kernel code modification	Kernel Integrity Protection (KIP)	Hardware
SIP disabling	Requires reboot (Theorem 1)	Hardware
Physical memory via `/dev/mem`	Device node absent on Apple Silicon	Hardware
DMA via Thunderbolt/PCIe	Per-device IOMMU, default-deny	Hardware
RDMA via Thunderbolt 5	RDMA status in attestation	Attestation

The residual attack surface is physical memory probing of LPDDR5x memory soldered directly into the Apple System-on-Chip (SoC) package. Desoldering the memory without destroying it is infeasible given the soldered package-on-package design.

6.3 Multi-Layer Attestation Architecture (FIG. 3)

Referring now to FIG. 3, the attestation architecture **300** comprises four independent verification layers, each verifying different aspects of the provider's security posture through different trust anchors.

6.3.1 Layer 1: Secure Enclave Attestation (302)

On first execution, the provider generates a P-256 ECDSA key pair within the Apple Secure Enclave co-processor **108** using the CryptoKit framework. The private key never leaves the Secure Enclave hardware; only an opaque handle is stored on disk.

The provider constructs an attestation blob containing the following fields, serialized as JSON with alphabetically sorted keys:

- `authenticatedRootEnabled`: Boolean—Authenticated Root Volume (Merkle-tree sealed system volume) intact
- `binaryHash`: SHA-256 hash of the running provider binary executable
- `chipName`: Apple Silicon chip identifier (e.g., "Apple M4 Max")
- `encryptionPublicKey`: Base64-encoded X25519 public key for end-to-end encryption
- `hardwareModel`: Machine model identifier (e.g., "Mac16,1")
- `osVersion`: Operating system version string
- `publicKey`: Base64-encoded raw P-256 public key (64 bytes: $X \| Y$)
- `rdmaDisabled`: Boolean—RDMA over Thunderbolt 5 is disabled
- `secureBootEnabled`: Boolean—full Secure Boot is active
- `secureEnclaveAvailable`: Boolean—Secure Enclave hardware is present
- `serialNumber`: Device serial number for cross-referencing with MDM
- `sipEnabled`: Boolean—System Integrity Protection is enabled
- `systemVolumeHash`: Cryptographic hash of sealed system volume snapshot
- `timestamp`: ISO 8601 timestamp for freshness verification

The attestation blob is hashed with SHA-256 and signed using the Secure Enclave's P-256 private key, producing a DER-encoded ECDSA signature.

The coordinator server **104** verifies the attestation by: (a) decoding the P-256 public key from the attestation blob; (b) computing SHA-256 over the original JSON bytes (preserving exact byte representation for cross-language compatibility); (c) verifying the DER-encoded ECDSA signature; and (d) enforcing minimum security requirements including `secureEnclaveAvailable=true`, `sipEnabled=true`, `secureBootEnabled=true`, and `authenticatedRootEnabled=true`.

Cross-language signature compatibility is achieved by: encoding JSON with alphabetically sorted keys in all implementations (Swift's `JSONEncoder` with `.sortedKeys`, Go's `map` marshaling which sorts keys alphabetically as of Go 1.12); preserving original JSON bytes on the coordinator side using raw message handling rather than re-serializing; and using consistent base64 encoding (standard alphabet, not URL-safe variant).

6.3.2 Layer 2: MDM SecurityInfo Verification (304)

Layer 2 addresses two limitations of Layer 1: (a) the coordinator cannot cryptographically prove that the P-256 key actually resides in the Secure Enclave (signatures generated by software-held keys are indistinguishable from Secure Enclave-generated signatures); and (b) the SIP status in the attestation blob is checked via a software command (`csrutil status`), which could theoretically be spoofed on a sufficiently compromised system.

The coordinator sends an MDM SecurityInfo command to the provider device via the MDM protocol. This command is processed by the operating system's MDM subsystem—not by the provider's application software—and returns independently verified security state including:

- `SIPEnabled`: Whether System Integrity Protection is active
- `SecureBootLevel`: Boot security level ("full" = only Apple-signed code at boot)
- `AuthRootVolume`: Whether the Authenticated Root Volume Merkle tree seal is intact
- `FDE_Enabled`: Whether FileVault full-disk encryption is active
- `RecoveryLock`: Whether Recovery Mode requires a password
- `FirewallEnabled`: Whether the application firewall is active

Proposition 1 (MDM Verification Circularity)**.** *Spoofing the MDM SecurityInfo response to report SIP as enabled when it is actually disabled would require modifying system frameworks located in* `/System/Library/`*. However, modifying those frameworks requires SIP to be disabled. Therefore, a provider cannot simultaneously have SIP disabled AND report it as enabled through the MDM subsystem. This creates a self-reinforcing verification property: the MDM verification is unforgeable precisely when it matters most.*

6.3.3 Layer 3: Apple Managed Device Attestation (306)

Layer 3 verifies hardware identity and provenance through Apple's Managed Device Attestation (MDA) system, which produces Apple-signed X.509 certificate chains that are unforgeable by software-only adversaries.

The protocol operates as follows:

1. During enrollment, the provider device generates a P-384 ECDSA key pair within the Secure Enclave, with the key marked as `HardwareBound=true` and `Attest=true` in the ACME configuration profile.
2. The device contacts Apple's attestation servers, which verify the device's identity and security properties using Secure Enclave hardware attestation.
3. Apple issues a DER-encoded X.509 certificate chain comprising:

- A leaf certificate containing device-specific information encoded in Apple-assigned OID extensions
- An intermediate certificate (Apple Enterprise Attestation Sub CA 1, P-384)
- A root certificate (Apple Enterprise Attestation Root CA, P-384, valid until 2047)

4. The leaf certificate contains the following Apple-assigned OID extensions (all prefixed with `1.2.840.113635`):
   - OID `100.8.9.1`: Device serial number
   - OID `100.8.9.2`: Device UDID
   - OID `100.8.10.1`: Operating system version
   - OID `100.8.10.2`: Secure Enclave OS (SepOS) version
   - OID `100.8.10.3`: Low-Level Bootloader (LLB) version
   - OID `100.8.11.1`: Freshness code (replay prevention and key binding)
   - OID `100.8.13.1`: SIP status (independently verified by Apple)
   - OID `100.8.13.2`: Secure Boot level
   - OID `100.8.13.3`: Kernel extension loading status
5. The coordinator verifies the certificate chain against the hardcoded Apple Enterprise Attestation Root CA public key and cross-references the serial number and security properties against the provider's self-reported attestation blob.

The trust anchor for Layer 3 is the Apple Root CA, not the operating system's MDM client. Forging an MDA certificate chain would require compromising Apple's certificate authority infrastructure, which is outside the threat model.

6.3.4 Layer 4: Continuous Challenge-Response (308, FIG. 9)

Referring now to FIG. 9, Layer 4 provides continuous assurance that the provider's security posture has not degraded since enrollment.

At periodic intervals (in one embodiment, every 5 minutes), the coordinator server **104** sends an attestation challenge to the provider comprising a random 32-byte nonce and a timestamp. The provider must, within a timeout period (in one embodiment, 30 seconds):

1. Check the current SIP status via the operating system command
2. Check the current Secure Boot status
3. Compute a signature: $\sigma = \mathrm{Sign}_{\mathrm{SE}}(\text{SHA-256}(nonce \| timestamp \| public_key))$
4. Return the signature, echoed nonce, public key, and current SIP and Secure Boot status

The coordinator verifies: (a) the nonce matches the sent value; (b) the public key matches the registered key; (c) the ECDSA signature is valid; (d) SIP is reported as enabled; and (e) Secure Boot is reported as enabled.

If SIP or Secure Boot is reported as disabled in any challenge response, the provider is immediately marked as untrusted and excluded from receiving inference requests. No grace period is provided, because by the SIP Runtime Immutability Theorem, SIP disabling requires a reboot that would disconnect the provider; therefore, a connected provider reporting SIP as disabled indicates either a compromised system or a mismatch warranting immediate exclusion.

Three consecutive signature verification failures also result in the provider being marked untrusted, distinguishing compromised providers from providers experiencing temporary connectivity issues.

6.3.5 Secure Enclave Key Binding via MDA Nonce (FIG. 4)

Referring now to FIG. 4, the SE Key Binding protocol **400** addresses a gap in the attestation architecture: while the ACME `device-attest-01` challenge (Layer 3) generates hardware-attested Secure Enclave

keys, those keys are stored in the platform-restricted keychain and are inaccessible to third-party applications. The `AllowAllAppsAccess` flag is silently ignored when `HardwareBound=true`. The provider's signing key (used in Layers 1 and 4) is therefore a different key, generated by the provider's own application code, and the coordinator has no cryptographic proof that this key resides in a genuine Secure Enclave.

The key binding protocol **400** solves this by leveraging the MDM `DeviceAttestationNonce` field:

Step 402: The provider generates a Secure Enclave P-256 key pair k and sends the public key pk_k to the coordinator during registration.

Step 404: The coordinator computes a nonce $n = \text{base64}(\text{SHA-256}(pk_k))$, binding the nonce to the provider's specific public key.

Step 406: The coordinator sends an MDM `DeviceInformation` command to the provider device with the `DeviceAttestationNonce` field set to n.

Step 408: The provider device receives the MDM command through the operating system's MDM subsystem and contacts Apple's attestation servers.

Step 410: Apple's attestation servers verify the device identity and generate a fresh MDA certificate chain. The `FreshnessCode` field in the leaf certificate (OID `1.2.840.113635.100.8.11.1`) is set to $\text{SHA-256}(n)$, where n is the nonce provided by the coordinator.

Step 412: The coordinator receives the MDA certificate chain and verifies it against the Apple Enterprise Attestation Root CA.

Step 414: The coordinator verifies that $\text{FreshnessCode} = \text{SHA-256}(n)$, where $n = \text{base64}(\text{SHA-256}(pk_k))$. This confirms that the device that generated the Apple-signed certificate chain is the same device associated with the provider's signing key pk_k.

The security argument is as follows: only a genuine device that successfully checks in with Apple via Apple Push Notification service (APNs) can generate an MDA certificate with a FreshnessCode matching the coordinator's nonce. A software-only adversary cannot forge an Apple-signed certificate chain. Combined with SIP enforcement (which blocks binary replacement) and Secure Boot (which blocks bootloader tampering), the protocol provides strong evidence that the provider's signing key is held in a genuine Apple Secure Enclave on genuine Apple hardware.

6.4 Combined Enrollment Protocol (FIG. 5)

Referring now to FIG. 5, the combined enrollment protocol **500** unifies all provider enrollment into a single configuration profile to reduce friction and ensure atomic enrollment across all attestation layers.

The configuration profile (in one embodiment, a `.mobileconfig` file) contains three payloads:

SCEP Payload 502 (`com.apple.security.scep`): Generates an RSA-2048 identity certificate via the Simple Certificate Enrollment Protocol. This certificate authenticates the device to the MDM server.

MDM Payload 504 (`com.apple.mdm`): Enrolls the device with the MDM server using the SCEP identity certificate. The AccessRights bitmask is set to 1041 (binary: `10000010001`), which grants only three capabilities:

- Bit 0: Inspect device (basic query)
- Bit 4: Query device information (model, OS, serial)
- Bit 10: Query security information (SecurityInfo command)

The following capabilities are explicitly denied by omission from the bitmask:

- Bit 1: Install/remove configuration profiles
- Bit 2: Device lock and passcode removal
- Bit 3: Erase all data on device
- Bits 5–9: Network, provisioning profile, and application management
- Bits 11–12: Device settings and app management

This minimal permission set ensures the MDM enrollment grants the coordinator only the verification capabilities needed for attestation, without conferring any ability to erase, lock, modify settings on, or install software on the provider device.

ACME Payload 506 (`com.apple.security.acme`): Initiates the `device-attest-01` challenge with the following specifications:

- `KeyType = ECSECPrimeRandom`, `KeySize = 384` (P-384 key)
- `HardwareBound = true` (key generated in Secure Enclave)
- `Attest = true` (Apple attestation requested)
- Subject CN = device serial number (for cross-referencing)
- `DirectoryURL` = ACME CA server address

The enrollment flow proceeds as follows: (1) the provider runs an installation script that detects the device serial number via system profiler; (2) the script requests a per-device profile from the coordinator; (3) the coordinator generates a combined profile with fresh UUIDs; (4) macOS displays the profile contents and permissions to the user for review; (5) on user approval, macOS processes all three payloads atomically; (6) the provider device checks in with the MDM server and initiates the ACME challenge; (7) Apple generates the MDA certificate chain.

Providers can unenroll at any time by removing the profile via System Settings, which immediately removes all associated certificates and MDM enrollment.

6.5 End-to-End Encryption (FIG. 8)

Referring now to FIG. 8, the end-to-end encryption system ensures that the coordinator server never observes plaintext inference prompts, even though it routes requests between consumers and providers.

For each inference request:

Step 804a: The coordinator generates an ephemeral X25519 key pair (sk_e, pk_e) using a cryptographically secure random number generator.

Step 804b: The coordinator generates a random 24-byte nonce.

Step 804c: The coordinator computes a shared secret via X25519 Diffie-Hellman key exchange between the ephemeral private key and the provider's registered X25519 public key.

Step 804d: The coordinator encrypts the inference request payload (containing the user's prompt) using XSalsa20-Poly1305 authenticated encryption with the shared secret and nonce:

$$ciphertext = \text{XSalsa20-Poly1305.Encrypt}(plaintext,\ nonce,\ shared_secret)$$

Step 804e: The coordinator transmits the encrypted payload to the provider, comprising: the ephemeral public key (base64-encoded), and the nonce concatenated with the ciphertext (base64-encoded).

Step 808a: The provider computes the same shared secret via X25519 Diffie-Hellman key exchange between its persistent X25519 private key and the received ephemeral public key.

Step 808b: The provider extracts the 24-byte nonce from the first 24 bytes of the ciphertext payload.

Step 808c: The provider decrypts the payload using XSalsa20-Poly1305 authenticated decryption, which also verifies the Poly1305 authentication tag to detect any tampering.

The provider's X25519 key pair is generated using a cryptographically secure random number generator and stored at a restricted path (in one embodiment, `~/.dginf/node_key` with file permissions `0600`). The public key is registered with the coordinator as part of the attestation blob and bound to the Secure Enclave identity via the encryption public key field.

Forward Secrecy Property: Because each request uses a fresh ephemeral key pair that is discarded after use, compromise of any single ephemeral key reveals only the corresponding single request. Compromise of the provider's long-lived X25519 key does not reveal past requests whose ephemeral keys have been discarded.

6.6 Provider Scoring and Selection Algorithm (FIG. 7)

Referring now to FIG. 7, the provider scoring algorithm **700** computes a composite score for each provider to select a suitable provider for each inference request. The scoring function incorporates six factors:

$$s = (1 - \ell) \cdot d \cdot \tau \cdot \rho \cdot w \cdot h$$

Factor 702—Load Factor (ℓ)**:** The ratio of pending requests to maximum concurrent requests (in one embodiment, 4). Providers with more pending requests receive lower scores, preventing overload.

Factor 704—Decode Throughput (d)**:** The provider's measured decode throughput in tokens per second, reported during registration benchmarking. This value reflects the provider's actual inference speed, which varies by hardware: M1 Base achieves approximately 68 GB/s memory bandwidth, while M4 Max achieves up to 546 GB/s, directly impacting decode throughput.

Factor 706—Trust Multiplier (τ)**:** A multiplier based on the provider's verification status:

- 1.0 for hardware-verified providers (all four attestation layers passed)
- 0.8 for self-signed providers (Layer 1 only)
- 0.5 for unverified providers

Factor 708—Reputation Score (ρ)**:** A composite reputation score in the range $[0, 1]$ computed as:

$$\rho = 0.4 \cdot r_j + 0.3 \cdot r_u + 0.2 \cdot r_c + 0.1 \cdot r_t$$

where r_j is job success rate (0.5 for new providers), r_u is $\min(uptime/24\text{h},\ 1.0)$, r_c is challenge-response pass rate, and r_t is a response time factor equal to 1.0 for average response times ≤ 1s, linearly degrading to 0.0 at ≥ 10s.

Factor 710—Warm Model Bonus (w)**:** A multiplier of 1.5 if the requested model is already loaded in the provider's GPU memory (as reported in heartbeat messages), or 1.0 otherwise. This factor accounts for the cold-start penalty of 10–30 seconds required to load model weights into GPU memory when the model is not cached.

Factor 712—Health Factor (h)**:** A composite factor derived from real-time system metrics reported in provider heartbeat messages:

$$h = (1 - m_p) \cdot (1 - c_u) \cdot t_m$$

where m_p is memory pressure from `vm_stat` (active + wired + compressed pages / total), c_u is CPU utilization (1-min load average / core count, clamped to $[0, 1]$), and t_m is thermal multiplier (1.0 at nominal, linearly decreasing to 0.25 at critical thermal state based on CPU speed limit percentage).

The system includes a hardware detection module that identifies the specific Apple Silicon chip family (M1, M2, M3, M4) and tier (Base, Pro, Max, Ultra), and maintains a lookup table of memory bandwidths. Notably, variants within the same chip tier may have different memory bandwidths based on GPU core count and corresponding memory channel configuration (e.g., a 40-core GPU variant of M3 Max has 400 GB/s bandwidth due to 16 memory channels, while a 30-core variant has 300 GB/s due to 12 channels). This hardware-level differentiation directly impacts expected inference throughput and is reflected in the decode throughput factor.

The provider scoring is computed per-request (not cached), ensuring that dynamic factors such as load, health, and thermal state are always current.

6.7 Request Lifecycle and Cancellation (FIG. 8)

The complete request lifecycle proceeds as follows:

Step 802: A consumer **102** submits an inference request to the coordinator **104** via HTTPS.

Step 804: The coordinator encrypts the request payload using per-request ephemeral E2E encryption as described in Section 6.5.

Step 806: The coordinator selects the highest-scoring provider using the algorithm described in Section 6.6. If no provider is immediately available, the request is enqueued (in one embodiment, with a maximum queue depth of 10 requests and a 30-second timeout).

Step 807: The coordinator transmits the encrypted request to the selected provider via the WebSocket connection **114**.

Step 808: The provider decrypts the request as described in Section 6.5.

Step 810: The provider's inference engine **110** processes the request within the hardened process.

Step 812: The provider transmits the inference response back to the coordinator. For streaming responses, the provider transmits individual tokens as Server-Sent Events (SSE) with a `[DONE]` sentinel marking completion.

Step 814 (Cancellation): If the consumer disconnects, times out (in one embodiment, after 600 seconds), or the coordinator detects an error condition, the coordinator sends a cancel message to the provider. The provider maintains a mapping of request identifiers to cancellation tokens and task handles. Upon receiving a cancel message, the provider's cancellation token is activated, which causes the asynchronous select mechanism in the streaming loop to drop the response stream. Dropping the stream closes the HTTP connection to the inference backend, which causes the inference engine to detect the disconnect and cease token generation immediately. This provides sub-second cancellation propagation from consumer disconnect through the coordinator to the inference engine, including immediate release of GPU compute resources.

6.8 Idle GPU Management (FIG. 10)

Referring now to FIG. 10, the idle GPU management system manages the lifecycle of the inference backend to optimize GPU memory utilization across the provider fleet.

The inference backend (whether in-process or subprocess) is spawned on-demand when the first inference request arrives. Health checks are performed at periodic intervals (in one embodiment, every 30 seconds).

After a configurable idle timeout period (in one embodiment, 10 minutes) with no inference requests, the backend process is terminated via SIGTERM, with a forced SIGKILL after a grace period (in one embodiment, 10 seconds) if the process does not exit gracefully.

Upon termination, GPU memory (Metal VRAM) is released by the operating system, and system memory used for model weights is reclaimed. This enables multiple providers to share the same hardware or frees resources for the owner's other tasks.

When a new inference request arrives after the backend has been terminated, the backend is respawned and the model is reloaded. To prevent restart storms (rapid repeated crashes), an exponential backoff strategy is employed: the first restart occurs after a 1-second delay, doubling to 2 seconds, then 4 seconds, capped at 5 seconds.

The warm model status is tracked in the provider registry and reported in heartbeat messages, enabling the scoring algorithm (Section 6.6) to apply the warm model bonus and preferentially route requests to providers that already have the requested model loaded.

6.9 Consumer-Selectable Trust Tiers

The system supports consumer-selectable trust tiers, allowing consumers to specify their minimum acceptable verification level for providers that handle their inference requests:

- **Tier 0 (none):** No attestation verification required.
- **Tier 1 (self_signed):** Layer 1 only—Secure Enclave-signed attestation blob verified by the coordinator.
- **Tier 2 (hardware):** All four layers—MDM SecurityInfo, Apple MDA certificate chains, Secure Enclave key binding via MDA nonce, and continuous challenge-response.

Tier 2 provides the following cryptographic guarantees: MDM SecurityInfo independently confirms SIP, Secure Boot, and Authenticated Root Volume status through the OS management subsystem; Apple MDA certificate chains signed by the Apple Enterprise Attestation Root CA prove genuine hardware identity; the SE Key Binding protocol (Section 6.3) cryptographically binds the provider's signing key to Apple-verified hardware; and periodic challenge-response confirms fresh security posture.

6.10 Payment and Billing System

The system includes a micro-denomination payment ledger using integer arithmetic to prevent floating-point precision loss. All monetary amounts are represented in micro-USD ($1\ \text{USD} = 1{,}000{,}000\ \mu\text{USD}$), providing a direct 1:1 mapping to blockchain tokens with 6 decimal places. In one embodiment, pricing is set at $0.50 per 1 million output tokens, with the provider receiving 90% and the platform receiving 10%.

Provider wallet keys are stored in the macOS Keychain (which is hardware-backed on Macs with Secure Enclave) with a fallback to a file-based store with restrictive permissions (0600). Automatic migration from file-based to Keychain-based storage is supported.

Settlement occurs via blockchain smart contracts (in one embodiment, pathUSD tokens on the Tempo blockchain), with batch settlement of accumulated payouts.

7 ALTERNATIVE EMBODIMENTS

The following alternative embodiments describe variations of the disclosed systems and methods that achieve similar results through different specific implementations. These embodiments are within the scope of the disclosed subject matter.

7.1 A1. Alternative Hardware Platforms

While the detailed description focuses on Apple Silicon Macs, the software access path elimination technique applies to any platform providing: (a) a kernel-level anti-debugger mechanism; (b) runtime code signing enforcement that blocks memory inspection; (c) a system-level integrity protection mechanism that is immutable at runtime; and (d) a hardware co-processor for cryptographic key generation and signing.

On ARM-based systems with TrustZone, the Secure Enclave's role could be fulfilled by a Trusted Application running in the Secure World. On systems with TPM 2.0 modules, the TPM could provide hardware-bound key generation and attestation signing.

7.2 A2. Alternative Attestation Mechanisms

The multi-layer attestation architecture may be implemented using alternative verification mechanisms at each layer:

- Layer 1 may use any hardware-bound signing mechanism, including TPM 2.0 attestation keys, ARM TrustZone attestation, or FIDO2/WebAuthn authenticators.
- Layer 2 may use any independent operating system query mechanism, including Windows Management Instrumentation (WMI) on Windows, or dbus-based queries on Linux.
- Layer 3 may use any manufacturer-signed device attestation mechanism, including Android SafetyNet/Play Integrity, or Windows Device Health Attestation.
- Layer 4 may use any periodic challenge-response protocol; the interval, nonce size, and hash function may vary.

7.3 A3. Alternative Encryption Schemes

The end-to-end encryption may use alternative authenticated encryption schemes, including: AES-256-GCM with ECDH key exchange using P-256 or P-384 curves; ChaCha20-Poly1305 with X25519 key exchange; or post-quantum key encapsulation mechanisms such as ML-KEM (FIPS 203) for quantum-resistant encryption.

7.4 A4. Alternative Inference Engines

The in-process inference architecture is not limited to Python-based frameworks. Alternative embodiments include: direct C/C++ FFI bindings to inference libraries (e.g., MLX C++ API, llama.cpp, ONNX Runtime), eliminating the need for a Python interpreter entirely; WebAssembly (WASM)-based inference engines running in a sandboxed runtime; or GPU compute shader-based inference executing directly via Metal or Vulkan APIs without a high-level framework.

7.5 A5. Alternative Scoring Algorithms

The six-factor scoring algorithm may be replaced with alternative selection mechanisms, including: neural network-based provider selection trained on historical performance data; auction-based mechanisms where providers bid on requests; geographic proximity-based routing for latency optimization; or round-robin with health-based filtering for simplicity.

7.6 A6. Alternative Key Binding Mechanisms

The SE Key Binding via MDA Nonce protocol may be adapted to alternative attestation ecosystems. On Android, the key binding could use Android Key Attestation certificate chains (signed by Google's root

CA) with a challenge nonce derived from the provider's signing key. On Windows, the binding could use the Windows Device Health Attestation service with a TPM-bound nonce.

7.7 A7. Subprocess Mode with Unix Domain Sockets

In an alternative embodiment, the inference engine runs as a subprocess rather than in-process, communicating via Unix domain sockets instead of TCP. The socket path includes the parent process ID to prevent hijacking (e.g., `/tmp/dginf-backend-{pid}.sock`). While this provides less isolation than in-process execution (the subprocess binary could theoretically be replaced), it avoids the TCP interception attack vector and supports inference engines that cannot be embedded via FFI.

7.8 A8. Request Anonymization

In an alternative embodiment, the system includes an OHTTP (Oblivious HTTP, RFC 9458) relay between the consumer and coordinator, combined with RSA blind signatures, to prevent the coordinator from linking consumer identity to request content. This achieves non-targetability, where even a compromised coordinator cannot direct specific requests to specific providers for the purpose of targeting a particular user.

7.9 A9. Multi-Device Inference

In an alternative embodiment, multiple provider devices are linked via high-speed interconnects (e.g., Thunderbolt 5 RDMA with sub-50 microsecond latency) to shard large models across multiple devices. The attestation system is extended to verify all participating devices in a shard group, and the encryption system is extended to support multi-party computation across the shard.

7.10 A10. Encrypted Model Execution

In an alternative embodiment, model weights are encrypted at rest and decryption keys are released only to providers that have passed all four attestation layers. The provider's hardened process decrypts the model weights in protected memory, performs inference, and wipes the decrypted weights upon completion. This extends the privacy guarantees from protecting user prompts to also protecting model intellectual property.

7.11 A11. General Private Compute

The access path elimination technique is not specific to inference. In alternative embodiments, the same hardened process architecture protects arbitrary computation, including: fine-tuning on private datasets; evaluation on proprietary benchmarks; confidential data pipeline processing; or any computation requiring privacy on third-party consumer hardware.

8 CLAIMS

8.1 Independent Claims

Claim 1. A computer-implemented method for enabling private AI inference on a provider computing device controlled by an adversarial third party, the method comprising:

(a) at startup of a provider process on the provider computing device, invoking an operating system kernel mechanism to permanently deny debugger attachment to the provider process for the lifetime of the process;

(b) verifying that the provider binary is code-signed with hardened runtime enforcement enabled and without entitlements permitting external memory inspection;
(c) verifying that a system integrity protection mechanism of the operating system is enabled, wherein the system integrity protection mechanism prevents disabling of the protections of steps (a) and (b) without rebooting the computing device, and wherein rebooting terminates the provider process;
(d) loading an AI inference engine directly within the address space of the provider process via foreign function interface bindings, such that model weights, tokenizer state, and intermediate computation activations reside within the provider process's protected memory space without any inter-process communication;
(e) receiving an encrypted inference request containing an encrypted user prompt;
(f) decrypting the encrypted inference request within the provider process using a cryptographic key held by the provider;
(g) executing AI inference on the decrypted user prompt within the provider process; and
(h) after completing inference, sanitizing memory buffers that contained the user prompt and inference output using volatile write operations followed by a memory fence.

Claim 2. A system for decentralized private AI inference, the system comprising:

a coordinator server comprising one or more processors and memory, the coordinator server configured to:

- receive inference requests from consumer devices via a network interface;
- maintain a registry of provider devices with their capabilities, trust status, and real-time health metrics;
- encrypt inference request payloads using per-request ephemeral key exchange, such that the coordinator does not retain the ability to decrypt past requests;
- select a provider device from the registry using a multi-factor scoring algorithm; and
- route encrypted inference requests to selected provider devices;

one or more provider devices, each provider device being an Apple Silicon computing device with a Secure Enclave co-processor, each provider device running a hardened inference agent configured to:

- deny debugger attachment at the operating system kernel level;
- execute AI inference within a single hardened process with no inter-process communication;
- generate a cryptographic attestation blob signed by the Secure Enclave co-processor, the attestation blob comprising system integrity status, hardware identity, and a hash of the running binary;
- respond to periodic challenge-response attestation from the coordinator; and
- decrypt inference request payloads and execute inference within protected memory;

attestation infrastructure comprising:

- a mobile device management server configured to independently query security configuration of provider devices through operating system management interfaces; and
- a certificate authority configured to validate device attestation certificate chains signed by a hardware manufacturer.

Claim 3. A computer-implemented method for multi-layer hardware attestation of a provider computing device in a decentralized inference network, the method comprising:

(a) receiving, at a coordinator server, a signed attestation blob from the provider computing device, the attestation blob signed by a P-256 ECDSA private key held in a hardware security co-processor of the provider computing device, the attestation blob comprising at least: a system integrity protection

status, a secure boot status, a hardware model identifier, a hash of the running provider binary, and a public key corresponding to the signing private key;

(b) verifying the ECDSA signature of the attestation blob using the included public key and a SHA-256 hash of the attestation blob bytes;

(c) independently verifying the system integrity protection status and secure boot status of the provider computing device by querying a mobile device management subsystem of the provider computing device, the mobile device management subsystem being separate from the provider application software;

(d) verifying a hardware manufacturer-signed certificate chain for the provider computing device, the certificate chain comprising device identity information and security state information encoded in manufacturer-assigned certificate extensions;

(e) cryptographically binding the provider's signing public key to the hardware manufacturer-verified device identity by:

  (i) computing a nonce as a cryptographic hash of the provider's signing public key;

  (ii) transmitting the nonce to the provider computing device via the mobile device management subsystem;

  (iii) receiving a manufacturer-signed certificate containing a freshness code derived from the nonce; and

  (iv) verifying that the freshness code in the manufacturer-signed certificate corresponds to the transmitted nonce; and

(f) periodically transmitting challenge-response attestation requests to the provider computing device and verifying responses signed by the hardware security co-processor, the challenge-response comprising a random nonce, a timestamp, and current system integrity protection status.

Claim 4. A computer-implemented method for selecting a provider in a decentralized AI inference network, the method comprising:

(a) maintaining, at a coordinator server, a registry of provider devices, each provider device having an associated hardware profile comprising chip family, chip tier, GPU core count, and memory bandwidth;

(b) receiving, from each provider device at periodic intervals, heartbeat messages comprising: a list of models currently loaded in GPU memory, a memory pressure value derived from active, wired, and compressed memory pages, a CPU utilization value derived from load average normalized by core count, and a thermal state derived from CPU speed limit percentage;

(c) for each incoming inference request, computing a composite score for each available provider device, the composite score being a product of:

  - a load factor inversely proportional to the provider's pending request count;
  - a decode throughput factor based on the provider's measured inference speed;
  - a trust multiplier based on the provider's attestation verification level;
  - a reputation score derived from weighted combination of job success rate, uptime rate, challenge-response success rate, and response time;
  - a warm model bonus applied when the requested model is already loaded in the provider's GPU memory; and
  - a health factor derived from real-time memory pressure, CPU utilization, and thermal state; and

(d) selecting the provider device with the highest composite score to serve the inference request.

8.2 Dependent Claims

Claim 5. The method of Claim 1, wherein step (a) comprises invoking the `ptrace` system call with the PT_DENY_ATTACH flag, which permanently prevents all ptrace-based debugging for the lifetime of the process, including from processes with root privileges.

Claim 6. The method of Claim 1, wherein step (c) further comprises verifying that the system integrity protection mechanism provides runtime immutability such that: if the system integrity protection mechanism is verified as enabled at process startup time t_0, the mechanism remains enabled at all times during the process lifetime, because the only mechanism to disable the system integrity protection requires a hardware reboot that terminates all running processes.

Claim 7. The method of Claim 1, wherein step (d) further comprises restricting the inference engine's import path to include only packages from a code-signed application bundle and the standard library, excluding system-level package directories, such that modification of the import path requires breaking the code signature, which is prevented by the system integrity protection mechanism.

Claim 8. The method of Claim 1, wherein step (e) comprises receiving a payload containing: a base64-encoded ephemeral X25519 public key; and a base64-encoded ciphertext comprising a 24-byte nonce concatenated with XSalsa20-Poly1305 authenticated ciphertext; and wherein step (f) comprises computing a shared secret via X25519 Diffie-Hellman key exchange between the provider's persistent private key and the received ephemeral public key, and decrypting using the shared secret with Poly1305 authentication tag verification.

Claim 9. The method of Claim 1, further comprising: prior to step (e), verifying that Remote Direct Memory Access (RDMA) over Thunderbolt is disabled on the provider computing device, and including the RDMA status in the signed attestation blob.

Claim 10. The method of Claim 1, further comprising: computing a SHA-256 hash of the running provider binary executable and including the hash in a signed attestation blob, enabling the coordinator to verify that the provider is running an expected version of the software.

Claim 11. The system of Claim 2, wherein the per-request ephemeral key exchange comprises: generating a fresh X25519 key pair for each inference request at the coordinator; encrypting the request payload using NaCl Box authenticated encryption (X25519 + XSalsa20-Poly1305) with the ephemeral private key and the provider's registered public key; and discarding the ephemeral private key after encryption, thereby providing forward secrecy.

Claim 12. The system of Claim 2, wherein the multi-factor scoring algorithm computes a health factor from real-time provider telemetry comprising: memory pressure derived from virtual memory statistics of the provider operating system; CPU utilization derived from load average normalized by the provider's CPU core count; and thermal state derived from CPU speed limit percentage, with thermal throttling reducing the provider's score proportionally.

Claim 13. The system of Claim 2, wherein the multi-factor scoring algorithm includes a warm model bonus that increases the score of provider devices that report having the requested model currently loaded in GPU memory, thereby preferentially routing requests to providers that can serve them without cold-start model loading delay.

Claim 14. The system of Claim 2, wherein the coordinator server maintains a request queue for inference requests that arrive when no provider device is immediately available, the queue having a configurable maximum depth and timeout, and wherein provider devices that complete an inference request automatically dequeue and serve waiting requests.

Claim 15. The method of Claim 3, wherein step (c) further comprises: verifying a self-reinforcing circularity property in which spoofing the mobile device management security information response to report the system integrity protection as enabled when it is disabled requires modifying operating system frameworks in a protected directory, which itself requires the system integrity protection to be disabled, such that the verification is unforgeable when the system integrity protection is enabled.

Claim 16. The method of Claim 3, wherein the hardware manufacturer-signed certificate chain of step (d) comprises: a leaf certificate containing device-specific OID extensions encoding serial number, device identifier, operating system version, secure enclave firmware version, bootloader version, freshness code, system integrity protection status, and secure boot level; an intermediate certificate signed by the hardware manufacturer; and a root certificate of the hardware manufacturer.

Claim 17. The method of Claim 3, further comprising: upon detecting that the system integrity protection status or secure boot status reported in a challenge-response is false, immediately marking the provider computing device as untrusted and excluding it from receiving inference requests without a grace period.

Claim 18. The method of Claim 3, further comprising: upon detecting three consecutive challenge-response signature verification failures, marking the provider computing device as untrusted.

Claim 19. The method of Claim 4, wherein the memory bandwidth for each provider device is determined from a lookup table indexed by chip family, chip tier, and GPU core count, and wherein variants within the same chip tier having different GPU core counts are assigned different memory bandwidth values reflecting different memory channel configurations.

Claim 20. The method of Claim 4, wherein the reputation score is computed as a weighted combination: 0.4 times the job success rate, plus 0.3 times the uptime rate capped at a 24-hour baseline, plus 0.2 times the challenge-response pass rate, plus 0.1 times a response time factor that is 1.0 for average response times of 1 second or less and linearly degrades to 0.0 for average response times of 10 seconds or more; and wherein new providers with no history receive a neutral reputation score of 0.5.

Claim 21. A computer-implemented method for enabling private computation on a provider computing device, the method comprising:

(a) establishing a hardened process on the provider computing device by: invoking an operating system kernel mechanism to deny external process inspection; verifying code signing with hardened runtime enforcement; and verifying that an operating system integrity protection mechanism prevents circumvention of the foregoing without a hardware reboot that terminates the process;
(b) loading a computation engine directly within the hardened process via foreign function interface bindings, eliminating inter-process communication;
(c) verifying the provider computing device's security posture through a multi-layer attestation architecture comprising at least: a hardware security co-processor signature, an independent operating system management query, and a hardware manufacturer-signed certificate chain;
(d) receiving encrypted computation inputs;
(e) decrypting and processing the computation inputs within the hardened process; and
(f) sanitizing memory after computation completes.

Claim 22. The method of Claim 21, wherein the computation engine performs large language model inference, fine-tuning on private datasets, evaluation on proprietary benchmarks, or confidential data pipeline processing.

Claim 23. A combined enrollment method for a decentralized compute network, the method comprising:

(a) generating a single device configuration profile containing:

- a certificate enrollment payload for generating a device identity certificate;
- a device management payload enrolling the device with a management server, the management payload specifying an access rights bitmask that grants only device inspection and security information query capabilities while explicitly denying device erasure, locking, settings modification, and application management capabilities; and
- a device attestation payload initiating a hardware manufacturer attestation challenge with a hardware-bound key generated in a hardware security co-processor;

(b) transmitting the configuration profile to the provider device;
(c) upon user approval, atomically processing all payloads in the configuration profile; and
(d) verifying that the certificate enrollment, device management enrollment, and hardware manufacturer attestation all complete successfully.

Claim 24. The method of Claim 23, wherein the access rights bitmask is 1041, granting only bit 0 (device inspection), bit 4 (device information query), and bit 10 (security information query).

ABSTRACT OF THE DISCLOSURE

A system and method for enabling private AI inference on decentralized consumer hardware controlled by adversarial third parties. The system achieves inference privacy without hardware Trusted Execution Environments by systematically eliminating every software access path through which the hardware owner could observe inference data. A hardened provider process is protected by kernel-level anti-debugger attachment, hardened runtime enforcement blocking external memory inspection, and operating system integrity protection that is formally proven to be immutable at runtime. An AI inference engine is embedded directly within the hardened process via foreign function interface bindings, eliminating inter-process communication attack surfaces. A four-layer attestation architecture independently verifies provider security through Secure Enclave signatures, mobile device management security queries, manufacturer-signed certificate chains, and continuous challenge-response verification. A protocol cryptographically binds provider signing keys to manufacturer-verified genuine hardware via an attestation nonce mechanism. The system supports per-request ephemeral end-to-end encryption providing forward secrecy, hardware-aware provider scoring with real-time health telemetry, and consumer-selectable trust tiers.

NOTES FOR ATTORNEY

1. **Drawings Required:** Ten figures as described in the Brief Description of Drawings section. These can be prepared as formal patent drawings from the system architecture diagrams in the accompanying paper and codebase.
2. **Priority Date Considerations:** The paper "Private Decentralized Inference on Consumer Hardware" is dated March 2026 and may constitute a public disclosure. The provisional should be filed as soon as possible to establish priority before the 1-year grace period from any public disclosure expires.
3. **Alice/Section 101 Strategy:** The claims are drafted to emphasize concrete technical improvements to computer security and hardware integration:
   - Claim 1 recites specific kernel-level mechanisms (ptrace, hardened runtime, SIP) and a formal immutability property
   - Claim 2 recites a specific multi-component system with hardware co-processors
   - Claim 3 recites a specific cryptographic key binding protocol with concrete steps
   - Claim 4 recites a concrete scoring algorithm with hardware telemetry inputs
   - Per the August 2025 USPTO memorandum on AI patent eligibility, claims describing technical improvements to computer functionality have strengthened eligibility arguments
4. **Prior Art to Address:**
   - Apple Private Cloud Compute (distinguishable: Apple owns hardware; the disclosed system assumes adversarial owner)
   - Intel TDX / AMD SEV-SNP (distinguishable: requires enterprise hardware; the disclosed system uses consumer devices)
   - Existing decentralized compute networks (distinguishable: none achieve inference privacy without TEEs)
   - FHE-based inference (distinguishable: impractical overhead; the disclosed system achieves real-time speeds)
5. **International Filing:** Consider PCT filing within 12 months for international protection. Key jurisdictions: US, EU, UK, Japan, South Korea (major AI infrastructure markets).
6. **Continuation Strategy:** The provisional is broad enough to support multiple continuation applications focused on: (a) the access path elimination method; (b) the multi-layer attestation architecture and key binding protocol; (c) the hardware-aware provider scoring system; and (d) the combined enrollment protocol.
7. **Entity Status:** Determine whether Eigen Labs qualifies for micro entity ($65) or small entity ($130) filing fees. Micro entity requires: fewer than 500 employees, neither applicant nor inventor named on more than 4 previously filed US patent applications, and gross income not exceeding $251,190 (2025 threshold).